

Exercício 01 — (4 pontos)

No Bolinha Restaurante, as comandas impressas dos pedidos agendados ficam organizadas em um varal físico, representando a sequência atual de acompanhamento da produção. Durante o expediente, essa sequência pode sofrer alterações sem que toda a organização precise ser refeita.

Quando um novo pedido agendado é liberado para entrar no varal com prioridade operacional, ele deve ocupar o início da sequência. Quando um pedido é cancelado, sua comanda precisa ser retirada do varal. Já quando ocorre erro de impressão ou correção de informações, a comanda deve ser atualizada na mesma posição em que já se encontra, sem perder seu lugar na sequência.

Implemente um programa em Python que solucione o problema de organização deste varal de comandas, garantindo que o sistema permita a navegação bidirecional e a manipulação rápida de pedidos em qualquer ponto da sequência.

Cada nó da estrutura deve representar uma comanda e armazenar os seguintes dados:

-> código, nome_cliente e descricao_pedido.

Implemente uma classe No e uma classe responsável por organizar o varal de comandas, seguindo o modelo trabalhado em sala de aula, com referências para o elemento anterior e para o próximo.

A classe deve possuir métodos que permitam:

1. Inserir uma nova comanda no final do varal.
2. Inserir um pedido agendado no início da lista.
3. Remover uma comanda cancelada a partir do código.
4. Alterar a descrição de uma comanda a partir do código do pedido, mantendo o nó na mesma posição da sequência.
5. Exibir o estado atual do varal na ordem em que as comandas aparecem.
6. Contar quantas comandas ainda estão ativas no varal.

Saídas esperadas

- A sequência atual das comandas no varal.
- A confirmação lógica de que a remoção ou a alteração foi realizada corretamente.
- A quantidade total de comandas presentes após as alterações.

Exemplo prático

Considere o varal inicial com as seguintes comandas:

101 - Marcos - Feijoada Completa

102 - Ana - Marmita de Bisteca

103 - Carla - Feijoada Pequena

Se um novo pedido agendado for inserido no início:

100 - João - Feijoada Grande

o varal passa a ser:

100 -> 101 -> 102 -> 103

Se a comanda de código 102 for cancelada, a sequência passa a ser:
100 -> 101 -> 103

Se a comanda de código 101 precisar ter sua descrição alterada para:
Feijoada Completa sem alho

a sequência continua sendo:

100 -> 101 -> 103

mas a comanda 101 passa a armazenar a nova descrição, sem mudança de posição no varal.

Para garantir a agilidade da cozinha, a solução deve priorizar operações de inserção e remoção com complexidade $O(1)$, evitando o deslocamento de elementos em memória, como ocorreria em um vetor comum ao reorganizar as comandas.

Toda a lógica deve estar concentrada nos métodos da classe, com foco na manipulação dos dados e na manutenção da posição dos nós na sequência. Implemente a solução de acordo com essas especificações.

Exercício 02 — (3 pontos)

Exercício 02 — Pilha (3 pontos)

No Bolinha Restaurante, a montagem da caixa de feijoada segue uma ordem fixa de camadas. Alguns itens são colocados primeiro para formar a base, enquanto outros ficam por cima para facilitar a retirada pelo cliente no momento da abertura da embalagem. Se um erro for identificado em um item que ficou abaixo do topo, não é possível acessá-lo diretamente sem retirar antes os itens colocados depois dele.

Nesse processo, o último item inserido é também o primeiro que pode ser retirado durante uma correção. Por isso, a solução deve respeitar exatamente a lógica de empilhamento usada na montagem da caixa.

Considere que os itens podem ser adicionados à caixa nesta ordem operacional:

- base: molhos, farofa, cumbuca de feijoada, feijao extra
- camada intermediária: bisteca, paio, torresmo, bacon
- camada superior: arroz, couve, mandioca, banana
- topo: laranja, caipirinha

Implemente um programa em Python para controlar a montagem e a correção dessa caixa, utilizando uma estrutura em que o último elemento inserido seja o primeiro a sair.

A estrutura deve permitir as seguintes operações:

1. Empilhar um novo item na caixa.
2. Desempilhar o item do topo.
3. Verificar se a estrutura está vazia.
4. Exibir o estado atual da caixa.
5. Corrigir a montagem até alcançar um item informado, retirando os itens necessários que estiverem acima dele.
6. Informar quantos itens foram removidos durante a correção.

Entradas do problema

- Nome do item a ser inserido.
- Nome do item que deve ser alcançado para correção.
- Chamadas aos métodos responsáveis pelas operações.

Saídas esperadas

- Item inserido no topo.
- Item removido.
- Estado atual da estrutura.
- Indicação se a estrutura está vazia.
- Quantidade de itens retirados até alcançar o item com problema.

Exemplo prático

Considere a caixa com a seguinte sequência de montagem:

molhos, farofa, cumbuca de feijoada, arroz, couve, laranja

Se for necessário corrigir o item arroz, os itens acima dele deverão ser retirados primeiro.

A sequência de remoção será:

laranja

caipirinha

Depois disso, o item arroz poderá ser retirado para correção.

Resultado esperado:

- foram removidos 2 itens antes de alcançar o item com problema;
- a estrutura restante mantém apenas os itens que estavam abaixo dele.

A implementação pode utilizar `collections.deque`. Toda a lógica deve estar concentrada em métodos da classe responsável pela estrutura.

Implemente a solução de acordo com essas especificações.

Exercício 03 — (3 pontos)

Exercício 03 — Fila (3 pontos)

No Bolinha Restaurante, a cozinha recebe pedidos de duas origens diferentes: SALÃO, quando o cliente faz o pedido presencialmente, e DELIVERY, quando o pedido é registrado para entrega. Embora a origem seja diferente, a cozinha deve preparar os pratos na ordem exata em que os pedidos foram realizados, para manter a organização e evitar conflitos no fluxo de produção.

Nesse contexto, a origem do pedido não altera sua posição na sequência de atendimento. Ela serve apenas para identificar o destino correto do prato após o preparo: pedidos do SALÃO devem seguir para montagem em louça, enquanto pedidos de DELIVERY devem seguir para embalagem. Assim, o problema consiste em manter uma sequência única de atendimento, preservando rigorosamente a ordem cronológica de entrada dos pedidos e permitindo identificar a categoria de cada um no momento da retirada.

Implemente um programa em Python para controlar essa sequência de pedidos da cozinha.

Cada pedido deve armazenar, no mínimo, os seguintes dados: código, descrição_pedido e origem, em que a origem pode ser apenas SALAO ou DELIVERY.

A estrutura deve permitir as seguintes operações:

1. Inserir um novo pedido no final da fila.
2. Atender o próximo pedido da fila.
3. Verificar se a fila está vazia.
4. Exibir o estado atual da fila.
5. Processar todos os pedidos restantes da fila.
6. Ao final do processamento, informar quantos pedidos eram do SALAO e quantos eram do DELIVERY.

Entradas do problema

- Dados do pedido a ser inserido na fila.
- Chamadas aos métodos responsáveis pelas operações da estrutura.

Saídas esperadas

- Confirmação do pedido inserido.
- Dados do pedido removido no atendimento.
- Exibição da fila na ordem atual.
- Indicação se a fila está vazia.
- Sequência final de atendimento dos pedidos restantes.
- Total de pedidos atendidos do SALAO.
- Total de pedidos atendidos do DELIVERY.

Exemplo prático

Considere a entrada dos seguintes pedidos, nesta ordem:

10 - Feijoadá Completa - SALAO

11 - Marmita de Bisteca - DELIVERY

12 - Feijoadá Pequena - SALAO

Se um pedido for atendido, o primeiro a sair deve ser:

10 - Feijoada Completa - SALAO

Se, em seguida, todos os pedidos restantes forem processados, a ordem final de atendimento será:

11 - DELIVERY

12 - SALAO

Ao final, o programa deve informar que, entre os pedidos processados nessa etapa:

- 1 pedido era do DELIVERY;
- 1 pedido era do SALAO.

A implementação pode utilizar `collections.deque`. Toda a lógica deve estar concentrada nos métodos da classe responsável pela estrutura, preservando a ordem de chegada dos pedidos.

Implemente a solução de acordo com essas especificações.